

UNIVERSITI TEKNOLOGI MALAYSIA
FACULTY OF COMPUTING

CHAPTER 9 LAB MANUAL

Building a RESTful API with PHP Slim 4

(In-memory Books API — no database yet)

FIELD	DETAILS
Course	SCSM2223 — Cross-Platform Application Development
Chapter	9 — Server-Side and PHP Slim
Lab time	2 hours
Lab assessment	4% (Lab Exercise component)
Software	Laragon, Composer, Sublime Text / VS Code, Postman
Companion	Reference solution: Ch9_BooksAPI_Solution/

1. Lab Objectives

By the end of this lab you will be able to:

1. Set up Laragon and Composer ready to build a Slim 4 application.
2. Scaffold a Slim 4 project with the recommended folder layout.
3. Define routes for GET, POST, PUT, and DELETE verbs on a single resource.
4. Read path parameters, query strings, and JSON request bodies.
5. Return JSON responses with the correct Content-Type and HTTP status codes.
6. Wire up custom PSR-15 middleware (JSON body parsing and CORS).
7. Test every endpoint with the VS Code REST Client (or Postman / curl).

2. Prerequisites & Setup Checklist

Before you start, verify each item below. Tick them off as you go.

#	Tool	Verify by running...	Expected result
1	Laragon (Full edition)	Open Laragon, click Start All	Apache + MySQL turn green
2	PHP 8.1+	php -v	PHP 8.1.x or newer
3	Composer	composer --version	Composer version 2.x
4	Code editor	Open Sublime Text or VS Code	Editor opens
5	REST client	Install "REST Client" extension in VS Code, or open Postman	Tool ready
TIP	If php or composer commands are not recognised, open the Laragon Terminal (right-click Laragon's tray icon → Terminal). It pre-loads Laragon's PHP/Composer onto the PATH.		

3. What We Will Build

A small RESTful API for a Book resource. There is NO database yet — the data lives in a PHP array, just like Chapter 9's slides. Chapter 10 will add MySQL via PDO.

Endpoints we will create:

Method	Path	Purpose	Status codes
GET	/	Health-check / API info	200
GET	/api/books	List all books (with optional ?q & ?limit)	200
GET	/api/books/{id}	Get one book by id	200, 404
POST	/api/books	Create a new book	201, 400
PUT	/api/books/{id}	Update a book (full or partial)	200, 400, 404
DELETE	/api/books/{id}	Delete a book	200, 404

Part A — Environment Setup

Step 1 — Pick your project folder

Open the Laragon Terminal and create the project folder inside Laragon's www directory:

```
cd C:\laragon\www
mkdir books-api
cd books-api
```

Step 2 — Initialise Composer

Composer is the PHP package manager. Initialise a new package — no questions asked:

```
composer init --name="utm/books-api" --no-interaction
```

This creates a composer.json file in your project root.

Step 3 — Install Slim 4 and the PSR-7 implementation

Slim 4 is the framework. slim/psr7 provides the concrete Request/Response objects that Slim works with.

```
composer require slim/slim:"4.*" slim/psr7
```

After this finishes, your folder will contain a vendor/ directory and an updated composer.json.

Step 4 — Confirm the install

```
dir vendor\slim\slim
composer show | findstr slim
```

CHECKPOINT You should see slim/slim and slim/psr7 listed. If composer show is empty, run composer install once more.

Part B — Scaffold the Project Structure

Create the following folders manually (or via mkdir):

```
books-api\  
├── public\  
│   ├── index.php      ← entry point  
│   └── .htaccess  
├── src\  
│   ├── routes.php  
│   ├── Controllers\  
│   │   └── BookController.php  
│   ├── Middleware\  
│   │   ├── JsonBodyParser.php  
│   │   └── Cors.php  
│   └── Data\  
│       └── books.php  
├── composer.json  
└── vendor\            (already created by Composer)
```

Step 5 — Add the PSR-4 autoload section

Open composer.json and add the autoload block so the App* namespace maps to src/:

```
{  
  "name": "utm/books-api",  
  "require": {  
    "slim/slim": "4.*",  
    "slim/psr7": "^1.6"  
  },  
  "autoload": {  
    "psr-4": { "App\\": "src/" }  
  }  
}
```

Then run:

```
composer dump-autoload
```

Part C — "Hello, Slim!"

Step 6 — public/index.php

Create the bootstrap file. This is the only file the web server hits directly.

```
<?php
use Psr\Http\Message\ResponseInterface as Response;
use Psr\Http\Message\ServerRequestInterface as Request;
use Slim\Factory\AppFactory;

require __DIR__ . '/../vendor/autoload.php';

$app = AppFactory::create();
$app->addRoutingMiddleware();
$app->addErrorMiddleware(true, true, true);

$app->get('/', function (Request $req, Response $res) {
    $res->getBody()->write('Hello, Slim!');
    return $res;
});

$app->run();
```

Step 7 — public/.htaccess

This file tells Apache to send every request to index.php so Slim can route it.

```
RewriteEngine On
RewriteCond %{REQUEST_FILENAME} !-f
RewriteCond %{REQUEST_FILENAME} !-d
RewriteRule ^ index.php [QSA,L]
```

Step 8 — Run it

Two ways to run the app:

Option A — Use Laragon's Apache. After starting Laragon, browse to:

```
http://localhost/books-api/public/
```

Option B — Built-in PHP server (handy for development). From the Laragon Terminal:

```
php -S localhost:8000 -t public
```

Then open <http://localhost:8000/>. You should see Hello, Slim!

TROUBLE? If you see a blank page or 500 error, check Laragon → Apache → Error Log. Most issues are missing vendor/ (run `composer install`) or wrong path in `index.php`'s `require`.

Part D — Build the Books Resource

Step 9 — Seed data: src/Data/books.php

```
<?php
return [
    ['id'=>1, 'title'=>'Clean Code',          'author'=>'Robert C. Martin', 'year'=>2008,
    'genre'=>'Software Engineering'],
    ['id'=>2, 'title'=>'Eloquent JavaScript', 'author'=>'Marijn Haverbeke', 'year'=>2018,
    'genre'=>'Programming'],
    ['id'=>3, 'title'=>'Vue.js 3 By Example', 'author'=>'John Au-Yeung',      'year'=>2021,
    'genre'=>'Web Development'],
];
```

Step 10 — src/Controllers/BookController.php

This is the brain of the API. Each public method maps to one route. Add it incrementally — start with index() and show() so you have something to test, then add create / update / delete.

```
<?php
namespace App\Controllers;

use Psr\Http\Message\ResponseInterface as Response;
use Psr\Http\Message\ServerRequestInterface as Request;

final class BookController
{
    private static array $books = [];

    private static function bootstrap(): void
    {
        if (self::$books === []) {
            self::$books = require __DIR__ . '/../Data/books.php';
        }
    }

    /** GET /api/books — supports ?q= and ?limit= */
    public function index(Request $req, Response $res): Response
    {
        self::bootstrap();
        $params = $req->getQueryParams();
        $items = self::$books;

        if (!empty($params['q'])) {
            $q = mb_strtolower((string)$params['q']);
            $items = array_values(array_filter($items, fn($b) =>
                str_contains(mb_strtolower($b['title']), $q) ||
                str_contains(mb_strtolower($b['author']), $q)
            ));
        }

        if (!empty($params['limit'])) {
            $items = array_slice($items, 0, max(1, (int)$params['limit']));
        }

        return $this->json($res, ['count' => count($items), 'data' => $items]);
    }
}
```

```

}

/** GET /api/books/{id} */
public function show(Request $req, Response $res, array $args): Response
{
    self::bootstrap();
    $id = (int)($args['id'] ?? 0);
    $book = $this->findById($id);
    return $book
        ? $this->json($res, $book)
        : $this->json($res, ['error' => "Book {$id} not found"], 404);
}

/* ...continued on the next page... */
}

```

Continue the same class with *create()*, *update()*, and *delete()*:

```

/** POST /api/books */
public function create(Request $req, Response $res): Response
{
    self::bootstrap();
    $body = (array)($req->getParsedBody() ?? []);
    $errors = $this->validate($body, requireAll: true);
    if (!empty($errors)) {
        return $this->json($res, ['errors' => $errors], 400);
    }
    $id = (max(array_column(self::$books, 'id') ?: [0])) + 1;
    $book = [
        'id'      => $id,
        'title'   => trim($body['title']),
        'author'  => trim($body['author']),
        'year'    => (int)$body['year'],
        'genre'   => trim((string)($body['genre'] ?? 'Uncategorised')),
    ];
    self::$books[] = $book;
    return $this->json($res, ['message' => 'Book created', 'data' => $book], 201)
        ->withHeader('Location', '/api/books/' . $id);
}

/** PUT /api/books/{id} – full or partial update */
public function update(Request $req, Response $res, array $args): Response
{
    self::bootstrap();
    $id = (int)($args['id'] ?? 0);
    $idx = $this->findIndexById($id);
    if ($idx === null) return $this->json($res, ['error' => "Book {$id} not found"], 404);

    $body = (array)($req->getParsedBody() ?? []);
    $errors = $this->validate($body, requireAll: false);
    if (!empty($errors)) return $this->json($res, ['errors' => $errors], 400);

    $current = self::$books[$idx];
    foreach (['title', 'author', 'genre'] as $k) {

```

```

        if (array_key_exists($k, $body)) $current[$k] = trim((string)$body[$k]);
    }
    if (array_key_exists('year', $body)) $current['year'] = (int)$body['year'];
    self::$books[$idx] = $current;
    return $this->json($res, ['message' => 'Book updated', 'data' => $current]);
}

/** DELETE /api/books/{id} */
public function delete(Request $req, Response $res, array $args): Response
{
    self::bootstrap();
    $id = (int)($args['id'] ?? 0);
    $idx = $this->findIndexById($id);
    if ($idx === null) return $this->json($res, ['error' => "Book {$id} not found"], 404);
    $deleted = self::$books[$idx];
    array_splice(self::$books, $idx, 1);
    return $this->json($res, ['message' => 'Book deleted', 'data' => $deleted]);
}

```

Finally, the small private helpers used above:

```

private function findById(int $id): ?array {
    foreach (self::$books as $b) if ($b['id'] === $id) return $b;
    return null;
}

private function findIndexById(int $id): ?int {
    foreach (self::$books as $i => $b) if ($b['id'] === $id) return $i;
    return null;
}

private function validate(array $body, bool $requireAll): array {
    $errors = [];
    $rules = [
        'title' => fn($v) => is_string($v) && trim($v) !== '',
        'author' => fn($v) => is_string($v) && trim($v) !== '',
        'year' => fn($v) => is_numeric($v) && (int)$v >= 1000 && (int)$v <=
(int)date('Y'),
    ];
    foreach ($rules as $f => $check) {
        if ($requireAll && !array_key_exists($f, $body)) { $errors[$f] = "$f is required";
continue; }
        if (array_key_exists($f, $body) && !$check($body[$f])) $errors[$f] = "$f is
invalid";
    }
    return $errors;
}

private function json(Response $res, mixed $data, int $status = 200): Response {
    $res->getBody()->write(json_encode($data, JSON_PRETTY_PRINT));
    return $res->withHeader('Content-Type', 'application/json; charset=utf-8')->
withStatus($status);
}

```


Step 11 — src/routes.php

```

<?php
use Psr\Http\Message\ResponseInterface as Response;
use Psr\Http\Message\ServerRequestInterface as Request;
use Slim\App;
use App\Controllers\BookController;

return function (App $app): void {
    $app->get('/', function (Request $r, Response $s) {
        $s->getBody()->write(json_encode([
            'name' => 'Books REST API',
            'version' => '1.0.0',
        ]));
        return $s->withHeader('Content-Type', 'application/json');
    });

    $app->group('/api', function ($g) {
        $g->get ('/books', [BookController::class, 'index']);
        $g->get ('/books/{id}', [BookController::class, 'show']);
        $g->post ('/books', [BookController::class, 'create']);
        $g->put ('/books/{id}', [BookController::class, 'update']);
        $g->delete('/books/{id}', [BookController::class, 'delete']);
    });
};

```

Step 12 — Wire routes into public/index.php

```

<?php
use Slim\Factory\AppFactory;

require __DIR__ . '/../vendor/autoload.php';

$app = AppFactory::create();
$app->add(new App\Middleware\JsonBodyParser()); // ← Step 13
$app->add(new App\Middleware\Cors());           // ← Step 14
$app->addRoutingMiddleware();
$app->addErrorMiddleware(true, true, true);

(require __DIR__ . '/../src/routes.php')($app);

$app->run();

```

Part E — Add Middleware

Step 13 — JsonBodyParser middleware

Without this, `$req->getParsedBody()` returns null when clients send `application/json`. This middleware decodes the body once for every JSON request.

```
<?php // src/Middleware/JsonBodyParser.php
namespace App\Middleware;

use Psr\Http\Message\ResponseInterface;
use Psr\Http\Message\ServerRequestInterface;
use Psr\Http\Server\MiddlewareInterface;
use Psr\Http\Server\RequestHandlerInterface;

final class JsonBodyParser implements MiddlewareInterface
{
    public function process(ServerRequestInterface $request, RequestHandlerInterface
$handler): ResponseInterface
    {
        if (stripos($request->getHeaderLine('Content-Type'), 'application/json') === 0) {
            $raw      = (string)$request->getBody();
            $decoded  = $raw === '' ? [] : json_decode($raw, true);
            if (!is_array($decoded)) $decoded = [];
            $request  = $request->withParsedBody($decoded);
        }
        return $handler->handle($request);
    }
}
```

Step 14 — Cors middleware

This middleware lets a Vue dev server (e.g. `http://localhost:5173`) call the API from the browser without being blocked by the Same-Origin Policy.

```
<?php // src/Middleware/Cors.php
namespace App\Middleware;

use Psr\Http\Message\ResponseInterface;
use Psr\Http\Message\ServerRequestInterface;
use Psr\Http\Server\MiddlewareInterface;
use Psr\Http\Server\RequestHandlerInterface;

final class Cors implements MiddlewareInterface
{
    public function process(ServerRequestInterface $request, RequestHandlerInterface
$handler): ResponseInterface
    {
        if ($request->getMethod() === 'OPTIONS') {
            $response = new \Slim\Psr7\Response(204);
            return $this->withCors($response);
        }
        return $this->withCors($handler->handle($request));
    }
}
```

```
private function withCors(ResponseInterface $r): ResponseInterface {  
    return $r  
        ->withHeader('Access-Control-Allow-Origin', '*')  
        ->withHeader('Access-Control-Allow-Headers', 'Content-Type, Authorization')  
        ->withHeader('Access-Control-Allow-Methods', 'GET, POST, PUT, DELETE, OPTIONS');  
}
```

Part F — Test the API

Restart the server (Ctrl+C, then re-run `php -S localhost:8000 -t public`). Then run each of the following requests.

Test 1 — Health check

```
GET http://localhost:8000/

Expected: 200 OK
Body: { "name": "Books REST API", "version": "1.0.0" }
```

Test 2 — List all books

```
GET http://localhost:8000/api/books

Expected: 200 OK
Body shape: { "count": 3, "data": [ { ... }, { ... }, { ... } ] }
```

Test 3 — Search and limit

```
GET http://localhost:8000/api/books?q=clean
GET http://localhost:8000/api/books?limit=2
```

Test 4 — Get one / not found

```
GET http://localhost:8000/api/books/1      → 200
GET http://localhost:8000/api/books/999    → 404 with { "error": "Book 999 not found" }
```

Test 5 — Create a book

```
POST http://localhost:8000/api/books
Content-Type: application/json

{ "title": "JavaScript Everywhere", "author": "Adam D. Scott", "year": 2020 }

Expected: 201 Created + Location: /api/books/4
```

Test 6 — Validation error

```
POST http://localhost:8000/api/books
Content-Type: application/json

{ "title": "Just a title" }

Expected: 400 with { "errors": { "author": ..., "year": ... } }
```

Test 7 — Update and delete

```
PUT http://localhost:8000/api/books/1
Content-Type: application/json

{ "year": 2009 }

DELETE http://localhost:8000/api/books/2
```

REMEMBER PHP processes do not share state between requests. After every restart, the data resets to the seed file. That is intentional — Chapter 10 introduces real persistence with MySQL.

Part G — Self-Check & Extension Tasks

Tick off each item once you can demonstrate it from your own running code (not just from the reference solution).

8. Show a 200 response with all four seed books.
9. Show a 404 response when requesting an id that does not exist.
10. Show a 201 response that includes a Location header pointing to the new resource.
11. Show a 400 response when validation fails, with field-by-field error messages.
12. Show CORS headers in the response when called from a different origin.
13. Refactor the inline closure in routes.php to use a controller method (e.g., for the health-check route).
14. BONUS — add a logging middleware that records the method, path, and response time for every request to a file.
15. BONUS — group all /api routes and add a stub AuthMiddleware that returns 401 unless the request has an Authorization header (real JWT comes in Chapter 11).

Appendix — Common Errors & Troubleshooting

Symptom	Likely cause	Fix
404 Not Found on every URL	Apache rewrite rules not active	Check public/.htaccess exists; ensure mod_rewrite is enabled (it is by default in Laragon).
500 Internal Server Error	Syntax error or missing autoload	Check Laragon → Apache → Error Log; run composer install; run composer dump-autoload.
getParsedBody() returns null	JsonBodyParser middleware not registered	Add \$app->add(new App\Middleware\JsonBodyParser()); BEFORE addRoutingMiddleware().
CORS error in browser	Cors middleware missing, or registered after the route ran	Make sure Cors is added in index.php; restart the server.
Class App\Controllers\BookController not found	Autoload not regenerated	Run composer dump-autoload.
Data resets on every request	Expected behaviour	PHP processes are short-lived. Persistence comes in Chapter 10 with MySQL via PDO.
Composer not recognised	PATH not set	Open the Laragon Terminal — it pre-loads PHP and Composer.

End of Lab Manual.